



## 1. Problem Title & Link

**Title:** LeetCode 724 : Find Pivot Index

**Link:** [LeetCode 724 - Find Pivot Index](#)

## 2. Problem Statement (Short Summary)

You are given an integer array `nums`.

Find the **pivot index** where:

sum of all elements on left side

=

sum of all elements on right side

Return the **leftmost pivot index**.

If no pivot exists, return:

-1

In simple terms:

- Stand at an index
- Add everything on the left
- Add everything on the right
- If both sums are equal → return that index

## 3. Examples (Input → Output)

### Example 1

Input:

`nums=[1,7,3,6,5,6]`

Output:

3

Explanation:

Index = 3

Left sum:

$1+7+3=11$

Right sum:

$5+6=11$

Both are equal.

### Example 2

Input:

`nums=[1,2,3]`

Output:



-1

Explanation:

No pivot exists.

### Example 3

Input:

nums=[2,1,-1]

Output:

0

Explanation:

Left sum:

0

Right sum:

$1 + (-1) = 0$

### 4. Constraints

$1 \leq \text{nums.length} \leq 10^4$

$-1000 \leq \text{nums}[i] \leq 1000$

Special restrictions:

- Return leftmost pivot index
- Must optimize beyond nested loops

### 5. Core Concept (Pattern / Topic)

#### Prefix Sum

### 6. Thought Process (Step-by-Step Explanation)

#### Brute Force

For every index:

1. Compute left sum
2. Compute right sum
3. Compare both

Time:

$O(n^2)$

Slow because sums are recalculated repeatedly.

#### Optimized Thinking



Observation:

Total Sum:

left + current + right

Therefore:

right = totalSum-left-current

Algorithm:

1. Calculate total sum
2. Traverse array
3. Calculate right sum using formula
4. Compare:

leftSum == rightSum

5. Return index if equal
6. Update left sum

## 7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[1,7,3,6,5,6]

At index = 3

1 7 3 6 5 6

|-----| X |-----|

Left=11

Right=11

Pivot:

index=3

## 8. Pseudocode (Language Independent)

totalSum=sum(nums)

leftSum=0

for i from 0 to n-1:

rightSum=totalSum-leftSum-nums[i]

if leftSum==rightSum:



```
return i
```

```
leftSum+=nums[i]
```

```
return -1
```

## 9. Code Implementation

### Python

class Solution:

```
def pivotIndex(self, nums):
```

```
    totalSum = sum(nums)
```

```
    leftSum = 0
```

```
    for i in range(len(nums)):
```

```
        rightSum = totalSum-leftSum-nums[i]
```

```
        if leftSum == rightSum:
```

```
            return i
```

```
        leftSum += nums[i]
```

```
    return -1
```

### Java

```
class Solution {
```

```
    public int pivotIndex(int[] nums) {
```

```
        int totalSum=0;
```

```
        for(int num:nums){
```

```
            totalSum+=num;
```

```
        }
```



```
int leftSum=0;

for(int i=0;i<nums.length;i++){

    int rightSum=
        totalSum-leftSum-nums[i];

    if(leftSum==rightSum){
        return i;
    }

    leftSum+=nums[i];
}

return -1;
}
```

## 10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:

- One pass for total sum
- One pass for checking pivot

Space Complexity:

$O(1)$

Reason:

- Uses only variables

## 11. Common Mistakes / Edge Cases

1. Including current element in left sum

Wrong:

left=current+previous

Current element should not belong to either side.

2. Updating left sum before comparison

Wrong order:

```
leftSum+=nums[i]
```

check condition

Correct:

check condition

```
leftSum+=nums[i]
```

### 3. Missing single element case

[5]

Answer:

0

### 4. Negative numbers handling

[2,1,-1]

## 12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[1,7,3,6,5,6]

Total Sum:

28

| i | nums[i] | leftSum | rightSum | Condition |
|---|---------|---------|----------|-----------|
| 0 | 1       | 0       | 27       | FALSE     |
| 1 | 7       | 7       | 20       | FALSE     |
| 2 | 3       | 10      | 17       | FALSE     |
| 3 | 6       | 16      | 11       | TRUE      |

Output:

3

## 13. Common Use Cases (Real-Life / Interview)

Financial systems:

- Balance income and expenses

Load balancing:

- Divide workloads equally

Analytics:



- Split datasets evenly

Game systems:

- Equal partition points

#### 14. Common Traps (Important!)

1. Off-by-one mistakes
2. Updating left sum too early
3. Recomputing sums repeatedly
4. Forgetting negative values

#### 15. Builds To (Related LeetCode Problems)

Progression:

- LC 724 → Find Pivot Index
- LC 560 → Subarray Sum Equals K
- LC 303 → Range Sum Query
- LC 238 → Product of Array Except Self
- LC 525 → Contiguous Array

#### 16. Alternate Approaches + Comparison

| Approach           | Time     | Space  | Notes                    |
|--------------------|----------|--------|--------------------------|
| Brute Force        | $O(n^2)$ | $O(1)$ | Repeated sum calculation |
| Prefix Array       | $O(n)$   | $O(n)$ | Uses extra memory        |
| Running Prefix Sum | $O(n)$   | $O(1)$ | Optimal                  |

#### 17. Why This Solution Works (Short Intuition)

The total sum already contains all elements. By maintaining a running left sum, the right sum can be computed instantly using:

$\text{rightSum} = \text{totalSum} - \text{leftSum} - \text{current}$

This avoids recalculating sums and finds the pivot efficiently.

#### 18. Variations / Follow-Up Questions

1. Find all pivot indices
2. Return pivot value instead of index
3. Find equilibrium index in a linked list
4. Split array into equal sum parts



5. Solve for dynamic streaming data
- 6.